

Department: Computer Science and Engineering

Course code: CSE-347 (Information System Analysis and Design)

Semester: Summer 2026

Section: 01



Hostel Management System

Final Project Report



Instructor: Dr. Hosney Jahan, Assistant Professor
Department: Computer Science & Engineering

Submitted by: Group: 06
Ratul Hasan Nirjon (2023-3-60-498)
Md. Shakib Al Hasan (2024-1-60-200)
Rakib Al Hasan (2024-1-60-143)
Anjuman Ara (2024-1-60-279)

Table of Contents

1. Background/Introduction	1
2. Literature Review	2-3
2.1. Web-Based Hostel Management Systems	2
2.2. Integrated Hostel Management	2
2.3. GPS and Geofencing for Attendance	2
2.4. Security Practices in Web-Based Systems	3
2.5. Research Gap and Relevance to the Project	3
3. Problem Statement	3
4. Objective	4
5. Proposed System/Methodology	4-11
5.1. Feasibility Analysis	4-7
5.2. System Overview and Architecture	7
5.3. Technology Stack	8
5.4. System Architecture and Modules	8-9
5.5. Database Design	10
5.6. Geo-Fenced Attendance Subsystem	10
5.7. Security Design	10
5.8. Development and Deployment Methodology	11
6. Requirement Analysis	12-14
6.1. Functional Requirements	12-13
6.2. Non-Functional Requirements	14
7. Diagrams	15-18
8. CPM	19-20
PERT	21
9. Screenshots of UI	22-27
Conclusion	28
Future Recommendations	28
References	28

1. Background/Introduction

Managing a student hostel is a complex task that involves coordinating numerous daily activities: student admissions, room allocation, attendance tracking, fee collection, complaint resolution, meal planning, visitor management, and parcel handling, among others. Traditionally, many educational institutions rely on manual, paper-based record-keeping systems that are time-consuming, error-prone, and difficult to maintain. These outdated methods lead to delays in information retrieval, miscommunication between staff and students, and a lack of real-time transparency for both administrators and residents.

With the rapid advancement of information technology, there is a growing need for an integrated digital platform that can streamline hostel operations, improve data accuracy, and enhance overall experience for all stakeholders. The Hostel Management System (HMS) presented in this project is a comprehensive web-based application designed to automate and centralize the core administrative functions of a student hostel. The system caters to three distinct user roles:

- **Manager** – the administrative head who oversees all operations, including student registration, room allocation, invoice generation, attendance oversight, complaint resolution, and staff management.
- **Staff** – front-desk personnel (caretakers, guards, cooks) who handle daily tasks such as registering visitors, receiving parcels, recording student returns, and marking their own attendance.
- **Student** – residents who can view their room details, submit complaints, check invoices, request mess-off periods, give feedback, mark their own attendance via GPS, and track parcels.

The system is built using modern open-source technologies: Python with the Flask web framework, MySQL/MariaDB for data persistence, and a responsive frontend with vanilla JavaScript, Chart.js for analytics, and Leaflet/OpenStreetMap for geo-fenced attendance marking. It is designed to be scalable, secure, and easy to deploy both locally and on cloud platforms.

2. Literature Review

2.1. Web-Based Hostel Management Systems

Existing hostel management research shows that manual records and fragmented administrative processes can cause data inconsistency, delays, and difficulty in managing room allocation, complaints, and student information. Recent web-based systems such as StaySmart [2] demonstrate how centralized databases and role-based access can improve hostel administration by bringing major activities into a single platform.

Other hostel management systems similarly focus on integrating room allocation, attendance, fee management, and administrative operations into web-based platforms [1], [3], demonstrating the value of centralized digital management.

2.2. Integrated Hostel Management

Recent systems increasingly combine multiple hostel operations rather than focusing on a single function. Features such as role-based access [5], room management, fee tracking, complaints, visitor management, and notifications help reduce manual work and improve transparency. For example, recent smart hostel systems integrate administrative and security functions through a centralized platform [3].

However, many systems provide only a subset of these functions or require additional infrastructure and services. This creates an opportunity for a low-cost system that combines the major daily operations within one platform.

2.3. GPS and Geofencing for Attendance

Traditional attendance methods are vulnerable to manual errors and proxy attendance. Geofencing-based attendance systems address this issue by checking whether a user's device is physically located within a predefined geographical boundary before accepting attendance. Recent research has demonstrated the use of GPS and geofencing to improve attendance accuracy while reducing dependence on specialized hardware [4].

This approach is particularly suitable for the HMS because students and staff can use their smartphones without requiring dedicated attendance hardware.

2.4. Security Practices in Web-Based Systems

Beyond functional design, secure implementation practices are essential for any system that stores personal and financial data. Established guidance on parameterized queries and input handling [6] informed the HMS approach to preventing SQL injection, while role-based access control principles [5] shaped the session-based, role-guarded routing used to separate Manager, Staff, and Student privileges.

2.5. Research Gap and Relevance to the Project

The reviewed systems demonstrate the benefits of digital hostel administration and geofenced attendance, but there is still a need for a simple, integrated, and low-cost system combining hostel administration, room allocation, invoices, complaints, mess management, visitor and parcel tracking, notifications, and GPS-based attendance.

The developed HMS addresses this gap by combining these functions in one role-based web application using open-source technologies.

3. Problem Statement

The existing hostel administration process for the target institution currently relies heavily on manual or semi-automated procedures. This creates several recurring problems:

1. Inefficient Record-Keeping – Student information, room allocations, and financial records may be distributed across paper files or separate records, making information retrieval and updating difficult.
2. Attendance Fraud – Traditional roll-call attendance is time-consuming and susceptible to proxy marking, resulting in unreliable attendance records.
3. Delayed Complaint Resolution – Complaints submitted verbally or through paper forms can be difficult to track and may remain unresolved.
4. Manual Invoice Generation – Preparing invoices and tracking payment status manually is repetitive and prone to errors.
5. Poor Communication – Information about invoices, complaints, parcels, visitors, and other events may not reach students efficiently.
6. Limited Management Visibility – Managers lack a centralized view of occupancy, attendance, outstanding payments, complaints, and other important activities.
7. Visitor and Parcel Tracking – Without a centralized digital log, tracking visitors and parcel collection becomes difficult.
8. Mess Management Problems – Manual mess-off requests and meal management can cause unnecessary administrative work and disputes.

These problems demonstrate the need for a centralized Hostel Management System that improves record accuracy, communication, transparency, and operational efficiency.

4. Objective

The primary objective of this project was to design, develop, and deploy a web-based Hostel Management System that automates and streamlines major hostel operations.

The specific objectives were to:

1. **Centralize hostel records** — Replace manual record-keeping with a unified, web-based platform accessible to managers, staff, and students.
2. **Enable role-based access** — Provide secure dashboards for Managers, Staff, and Students with tailored functionalities.
3. **Digitize core operations** — Manage student registration, room allocation, invoices, complaints, mess management, visitor/parcel tracking, and attendance through integrated modules.
4. **Implement secure GPS attendance** — Use geo-fencing to ensure reliable attendance marking for students and staff.
5. **Ensure security and scalability** — Protect user data with password hashing, parameterized queries, and HTTPS deployment, while keeping the system cost-effective using open-source technologies.

5. Proposed System/Methodology

5.1. Feasibility Analysis

5.1.1. Legal & Ethical Feasibility

- **Licensing Compliance:** The system is built using open-source components, including Flask (BSD-3-Clause), Python (PSF), Leaflet (BSD-2-Clause), Chart.js (MIT), and MySQL/MariaDB (GPL). These technologies were selected to support compliance with applicable software licensing requirements.
- **Data Privacy Ethics:** Personal resident information, including NID/Passport information and contact details, is stored in secured, normalized database tables. Access to sensitive disciplinary and payment information is restricted to the respective student and authorized management personnel.
- **Academic Integrity:** The system was developed by the project team for the CSE347 course. External materials and open-source technologies used in the project are properly acknowledged and attributed.

5.1.2. Technical Feasibility

Hardware Requirements: any system capable of running Python 3 and MySQL/MariaDB; minimum 2 GB RAM (4 GB recommended); Intel Core i3 or equivalent; 100 MB free storage; standard LAN/Wi-Fi; a GPS-enabled smartphone for geo-fenced attendance.

Software Requirements: Windows/Linux/macOS; Python 3.10+; Flask 3.x with Jinja2; MySQL 8.0+/MariaDB 10.4+; mysql-connector-python; any modern browser; Chart.js, Leaflet+OpenStreetMap, Gunicorn; XAMPP, VS Code and Git for development.

Technology Assessment: Python+Flask provides a mature, well-documented environment where the same Jinja2-rendered codebase runs both locally and on the web; MySQL/MariaDB supports triggers, foreign keys and referential integrity across an 18-table, 3-trigger schema derived from the UML class design; Leaflet/OpenStreetMap gives cost-free geofencing without paid map API keys; Chart.js renders responsive attendance analytics; and the system is deployed via Render (web service) and Aiven (MySQL) to minimize hosting cost using free-tier services.

Security Assessment: passwords are hashed using bcrypt and never stored in plaintext; session-based authentication with role-guarded routes restricts access by Manager/Staff/Student role; database operations use parameterized queries; strict SQL mode rejects invalid data; database socket timeouts prevent hung connections; and geo-fencing uses browser geolocation over HTTPS in production (localhost is supported for development).

5.1.3. SCHEDULE FEASIBILITY

Estimated Timeline

The project timeline follows the critical path and was estimated using the PERT three-point technique (Optimistic, Most Likely, Pessimistic); full network diagrams and float tables are provided in Section 8 (CPM/PERT). The total estimated time was approximately 60 days (≈ 2 months).

Project Phase/Task	Estimated Time
Project Planning & Scope Definition	2–4 days
Requirements Gathering & SRS	5–11 days
Database Design & Implementation	3–5 days
Development Environment Setup (Python, Flask, MySQL, XAMPP)	1–3 days
UI/UX Design Prototype	6–10 days
Authentication & Authorization Module	5–9 days
Core Modules (Student, Attendance, Invoice, Mess, Visitor/Parcel, Complaint)	20–30 days
Frontend Integration & Styling	4–7 days
Testing, Security Validation & Bug Fixing	7–13 days
Deployment, Documentation & User Training	3–7 days
Total Estimated Time (PERT expected)	≈ 60 days (~ 2 months)

Table-1: Project Estimated Timeline

5.1.4. ECONOMIC FEASIBILITY

The economic feasibility analysis evaluated the estimated development and hardware costs of the Hostel Management System and the potential savings resulting from automation.

Development Costs

Cost Item	Amount (₹)
Software stack (Python, Flask, MySQL, Chart.js, Leaflet, VS Code)	0 (open source)
System analysis & design (6 weeks)	5,000
Programming & module development (~8 weeks)	15,000
UI/UX design	3,000
Testing, bug fixing & documentation	2,000
Total Development Cost	25,000

Table-2: Project Development Costs

Hardware Operational Costs:

Cost Item	Amount (₹)
Development laptop — 20% project allocation of a ₹60,000 device	12,000
Web server (Render free tier — no dedicated hardware)	0
Domain name (.com)	800/year
SSL/HTTPS (Let's Encrypt — free)	0
Electricity & internet share	12,000/year
Part-time maintenance labor	24,000/year
Estimated annual operational cost	36,800/year

Table-3: Project Hardware Costs

Cost-Benefit Summary: Estimated annual savings from paper/stationery elimination, administrative-hour reduction, faster fee collection, and reduced complaint-resolution time range from ₹35,000 to ₹70,000+. The one-time project cost is ₹37,000 (₹25,000 development + ₹12,000 hardware) with ₹0 recurring cost on the current free hosting tiers; even if paid hosting becomes necessary (~₹38,520/year), total cost remains below the estimated annual savings, giving a payback period of approximately 1 to 1.5 years.

5.2. System Overview and Architecture

The developed Hostel Management System is a full-stack web application following a client-server architecture. The backend uses Flask, which handles routing, business logic, authentication, and database interaction. The frontend uses HTML5, CSS3, vanilla JavaScript, and Jinja2 templates to provide responsive web interfaces.

The system uses a normalized MySQL/MariaDB database containing 18 tables for users, students, staff, hostels, rooms, attendance, invoices, complaints, notifications, visitors, parcels, and other hostel-related entities. The application is designed to work on both desktop and mobile browsers.

5.3. Technology Stack

Component	Technology
Backend	Python 3.10+, Flask 3.x
Database	MySQL 8.0+ / MariaDB 10.4+
Database Driver	mysql-connector-python
Frontend	HTML5, CSS3, JavaScript, Jinja2
Analytics	Chart.js
Maps / Geolocation	Leaflet + OpenStreetMap
Local Development	XAMPP, VS Code, Git
Deployment	Render + Aiven
Monitoring	UptimeRobot

Table-4: Project Technology Required

5.4. System Architecture and Modules

5.4.1. Authentication and Authorization

The system provides secure login for Manager, Staff, and Student users. Passwords are stored using scrypt hashing, while session-based role control restricts users to the functions permitted for their roles. Logged-in users can also change their passwords.

5.4.2. Manager Module

The Manager has access to:

- student registration and management;
- hostel and room management;
- gender-compatible room allocation;
- student and staff attendance management;
- invoice generation and payment tracking;
- complaint and feedback management;
- mess menu and mess-off management;
- violation management;
- visitor and parcel records;
- staff management;
- notifications and dashboard analytics.

The Manager dashboard provides summarized information such as students, rooms, complaints, unpaid invoices, violations, and attendance statistics.

5.4.3. Staff Module

Staff can:

- register student visitors;
- receive and manage parcels;
- record returning students;
- mark their own attendance;
- view their attendance history and statistics.

5.4.4. Student Module

Students can:

- view their profile, hostel, and room information;
- view invoices and payment status;
- submit complaints and feedback;
- request mess-off periods;
- view the weekly mess menu;
- record hostel in/out activities;
- mark attendance using GPS;
- view attendance history and statistics;
- view and collect received parcels.

5.4.5. Notification System

The system provides an in-app notification mechanism for important events such as new invoices, complaint updates, parcel arrivals, visitor events, mess-off decisions, violations, room allocation, and manager-generated notices. Users can view unread notifications and filter them by category.

5.4.6. Reporting and Printing

Managers can print individual invoices and student statements in a print-friendly format. Attendance information is also presented through dashboards and graphical statistics.

5.5. Database Design

The database consists of 18 normalized tables derived from the UML class design (Fig-1). A base user's table supports the three user types through related Manager, Staff, and Student tables. The database includes tables for: users and role-specific information; hostels and rooms; student and staff attendance; invoices; complaints and feedback; mess-off requests and menus; visitors and parcels; violations; student in/out records; and notifications.

Foreign keys and database constraints maintain data integrity. Triggers are used to automatically return a room bed to the available pool when a student is deleted and to prevent room allocation that violates hostel gender restrictions.

5.6. Geo-Fenced Attendance Subsystem

A key feature of the developed system is GPS-based geo-fenced attendance. Each hostel can have a defined latitude, longitude, and attendance radius. When a student or staff member marks attendance, the browser obtains the device's GPS coordinates. The server then calculates the distance between the user's location and the hostel location using the Haversine formula.

If the calculated distance is within the configured hostel radius, the attendance can be recorded as Present. Otherwise, the attendance request is rejected.

The system also stores attendance history and provides graphical statistics for reviewing attendance patterns. This approach provides location-based verification without requiring dedicated attendance hardware. The use of geofencing is consistent with recent attendance-management research that uses GPS boundaries to improve attendance reliability.

5.7 Security Design

- Several security mechanisms were implemented:
- Script password hashing prevents plaintext password storage;
- Role-based route protection prevents unauthorized access;
- Parameterized SQL queries reduce SQL injection risks;
- Strict SQL mode helps prevent invalid database values;
- Server-side geo-fence validation prevents attendance decisions from relying only on client-side checks;
- HTTPS/TLS protects communication in the deployed environment.

These measures are part of the implemented system rather than only planned security requirements.

5.8 Development and Deployment Methodology

The system was developed incrementally through the following stages:

- **Requirement Analysis** – Functional and non-functional requirements were identified.
- **System Design** – UML use case, class, and sequence diagrams were prepared, followed by database and interface design.
- **Technology Selection** – Flask, MySQL/MariaDB, and supporting technologies were selected based on the feasibility analysis.
- **Implementation** – Authentication and core management modules were developed first, followed by attendance, invoices, complaints, mess, visitor/parcel, and notification features.
- **Testing** – Individual modules, integrated functionality, user roles, and security controls were tested.
- **Deployment** – The completed application was deployed using Render and Aiven with TLS configuration.
- **Maintenance** – The modular structure allows future improvements such as pagination, PDF attendance reports, and password recovery.

The application currently supports both local development through XAMPP and cloud deployment through Render and Aiven.

6.1. Functional Requirements

1. User Authentication

- Students, Staff, and Manager (Admin) can log in to the system.
- Users can change their passwords.

2. Student Management & Discipline

- Manager can register a new student and delete student records.
- Students can view their profile.
- Manager can record and view student violations, send violation notices, and resolve violations.

3. Hostel & Room Management

- Manager can add, edit and delete hostel information.
- Hostels are gender-separated (Male/Female); the system refuses opposite-gender room allocation.
- Manager can add rooms with a custom bed capacity, allocate rooms to students, and view available rooms and each room's occupants.
- Students can view their room and hostel details.

4. Complaint Management

- Students can submit complaints mapped to their room.
- Manager can view, review, and update the status of complaints.

5. Invoice & Payment Management

- Manager can generate invoices (room rent/electricity/food/water/other), including several at once.
- Students can view invoice details; Manager can update billing and payment status.
- Managers and students can print individual invoices and per-student statements.

6. Attendance Management

- Students and staff can mark attendance with geo-fenced location (inside the hostel's GPS radius).
- Managers can record and view student and staff attendance history and edit a specific person's attendance day by day.
- The system shows attendance analytics (today's status donut + 14-day trend) and automatically records Absent for any day that passes unmarked (background backfill).

7. Staff Management

- Manager can add and remove staff.
- Staff can mark their own geo-fenced attendance, register visitors, receive parcels, and record student returns.

8. Mess Management

- Students can apply for Mess Off for date ranges; Manager can approve/view requests.
- Students can view the weekly menu; Manager can update it.

9. Visitor Management

- Staff can register student visitors at the front desk; Manager can view visitor tracking records.

10. Suggestions & Feedback

- Students can submit feedback records; Manager can view suggestions.

11. Student In/Out Management

- Students can record their in/out (leave) information; Staff can mark student returns; Manager can monitor entry/exit records.

12. Parcel Management

- Staff can track arriving parcels for students; students can track collection status and collect their parcels with a one-click action (with audit trail).

13. Notification System

- The system sends automatic alerts for complaint updates, new invoices, parcel arrivals, visitor arrivals, violation notices, and mess-off decisions.
- Managers can send free text notices to students and staff; users see an unread-count badge and can filter notifications by category.

14. Reporting & Printing

- Manager can print per-student invoice statements and per-invoice print copies; the system exposes a GET /healthz endpoint for up-time monitoring.

6.2. Non-Functional Requirements

1. Operational Requirements

- The system operates as a web-based application accessible via modern web browsers.
- The system can be deployed to free hosting (Render + Aiven free MySQL) with TLS-encrypted database connections.
- The system supports one-time schema/seed import and on-demand data synchronization between the local and web databases.

2. Performance Requirements

- The system retrieves and displays available rooms, room allocation, and the weekly mess menu in 2 seconds or less.
- The system updates billing and payment status within 3 seconds of the manager's confirmation.

3. Security Requirements

- Only authenticated users can access their respective dashboards after valid login.
- Only a manager can generate invoices or delete student records.
- All user passwords must be securely hashed using bcrypt; all database queries must use parameterized statements.
- Student disciplinary and payment history must be visible only to the respective student and authorized management.
- Attendance geo-fencing requires a secure (HTTPS) connection on deployed servers; localhost always works.

4. Cultural and Political Requirements

- No special cultural and political requirements are anticipated.
- Hostel accommodation respects gender separation as part of room allocation rules.

7. Diagrams

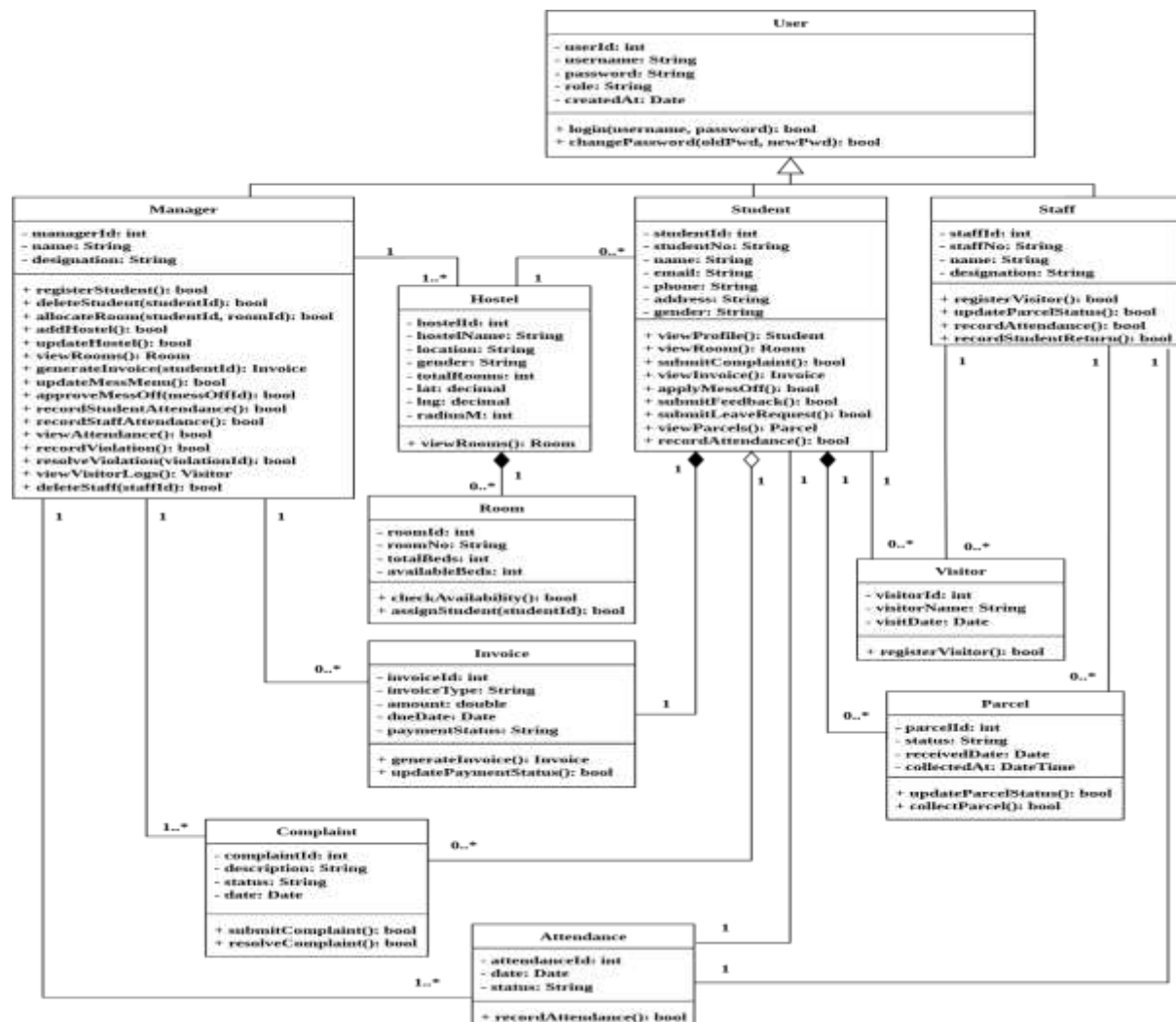


Fig-1: UML Class Diagram of the Hostel Management System

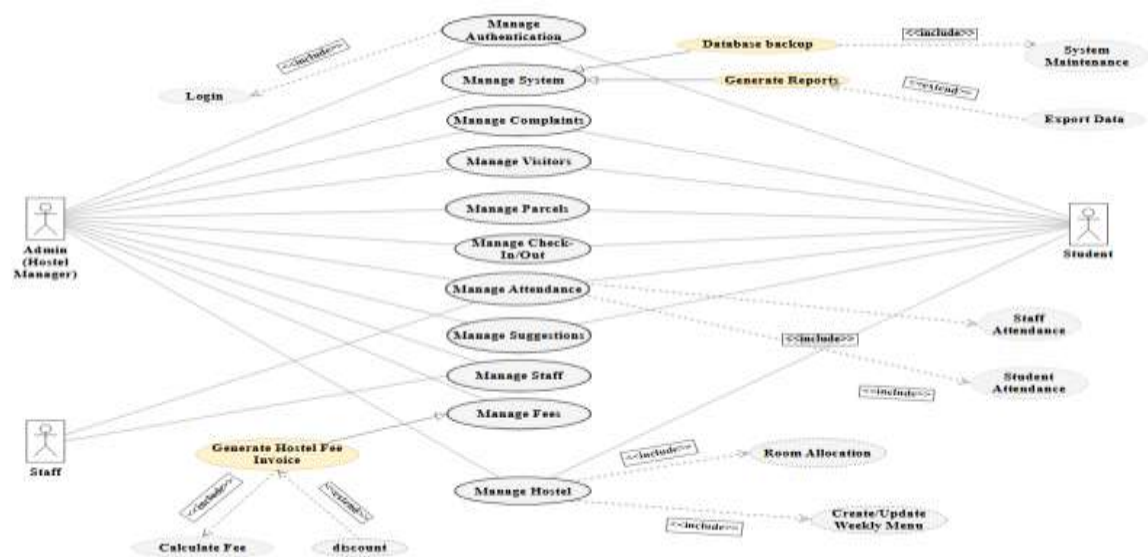


Fig-2: Use-Case Diagram of the Hostel Management System

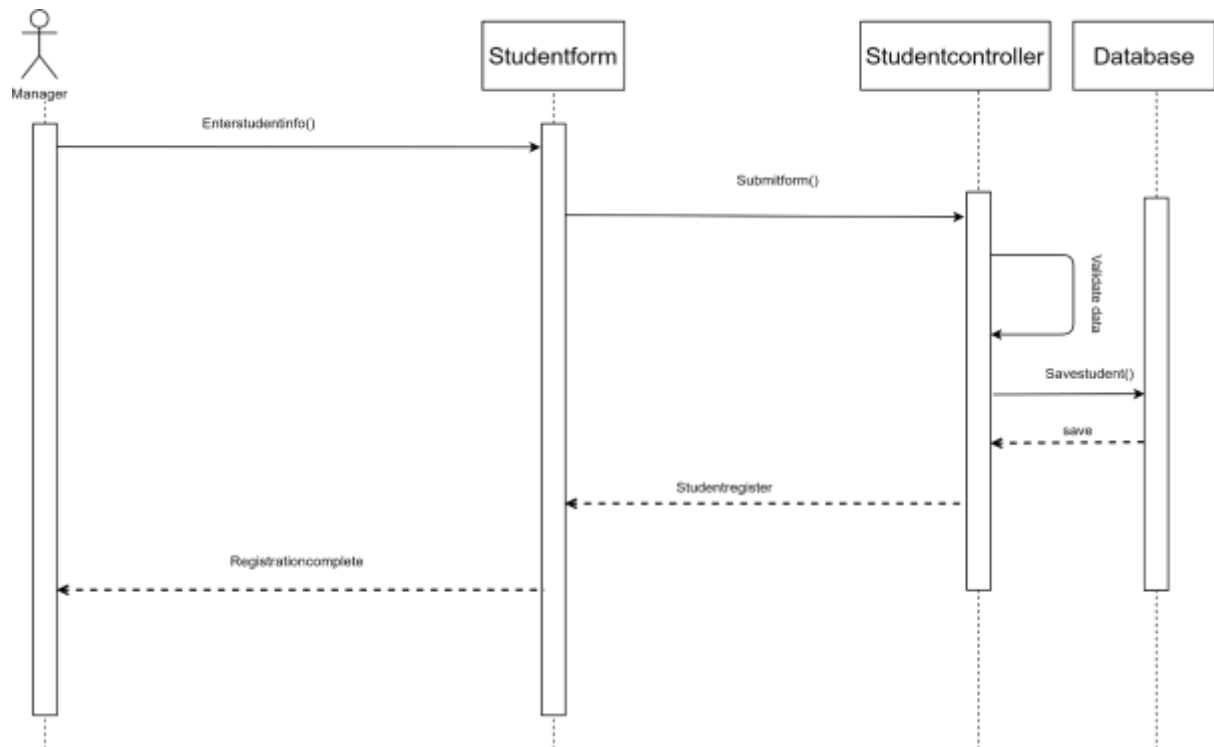


Fig-3: Sequence Diagram: Register New Students

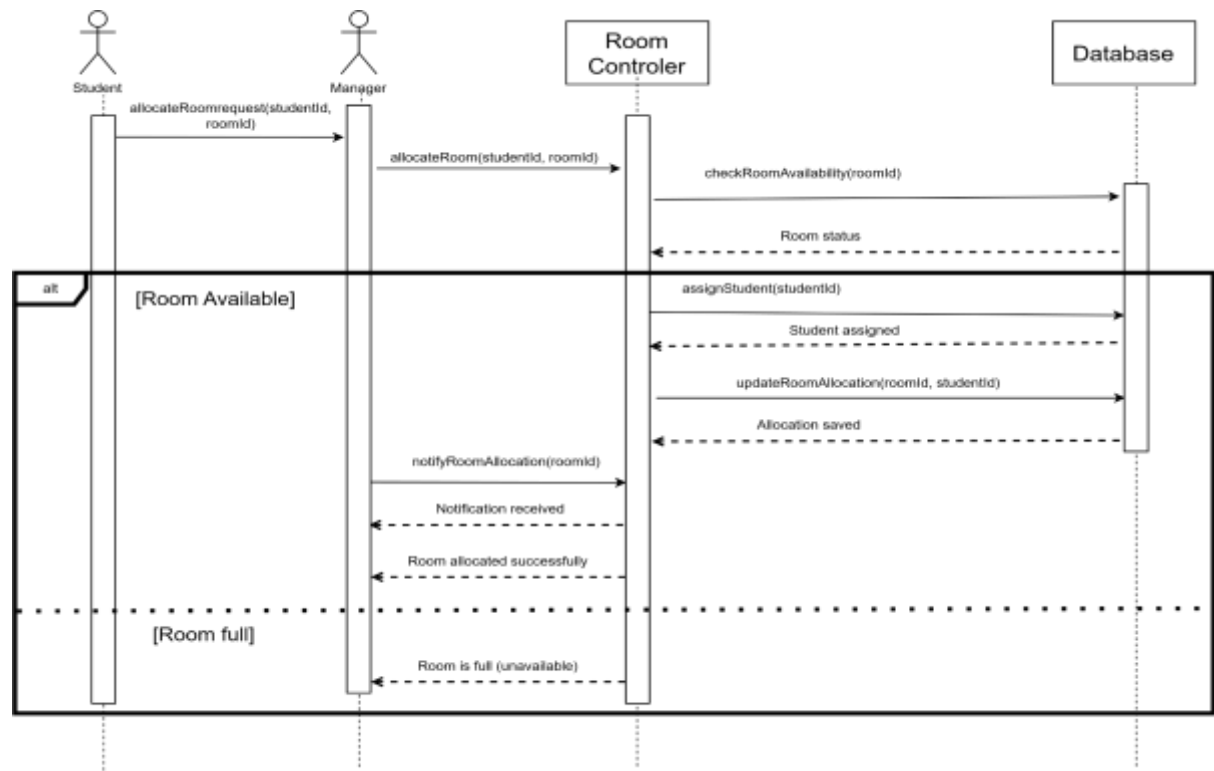


Fig-4: Sequence Diagram: Room Allocation

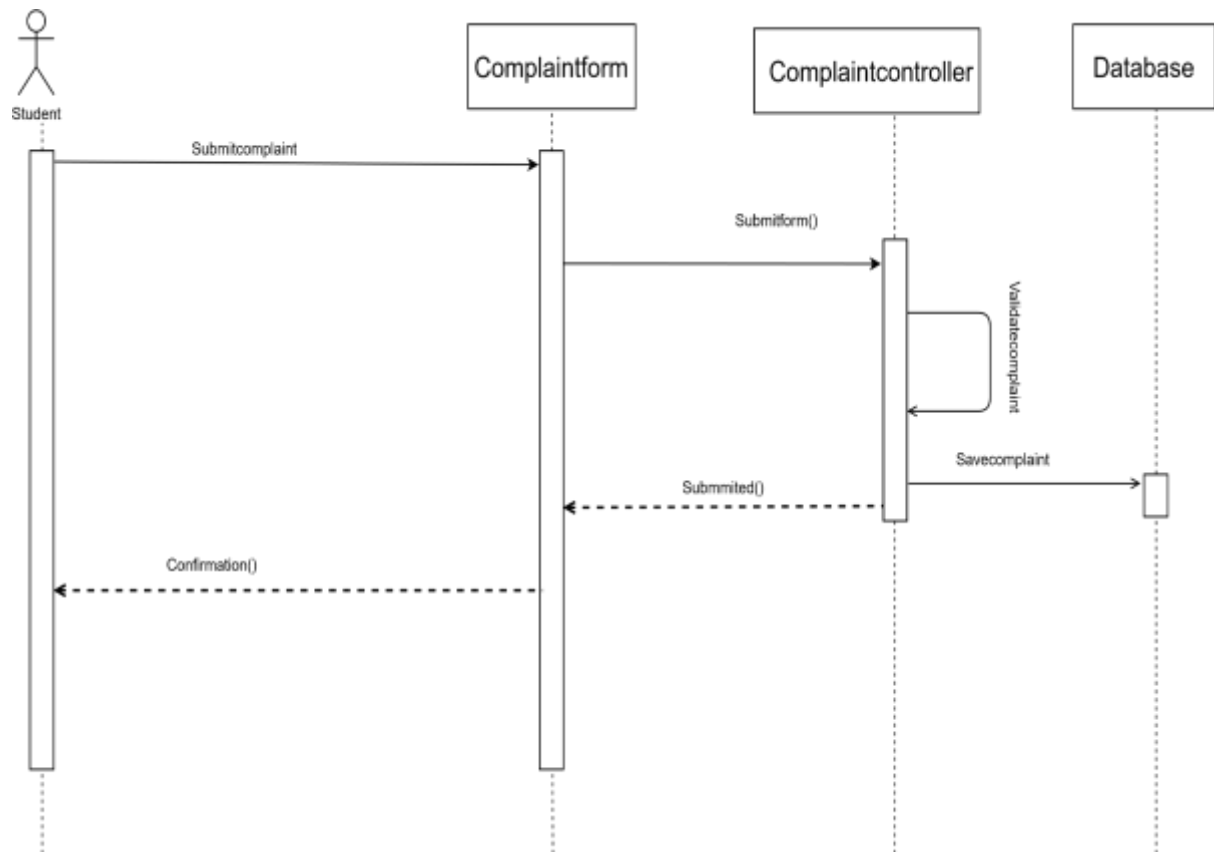


Fig-5: Sequence Diagram: Complaint

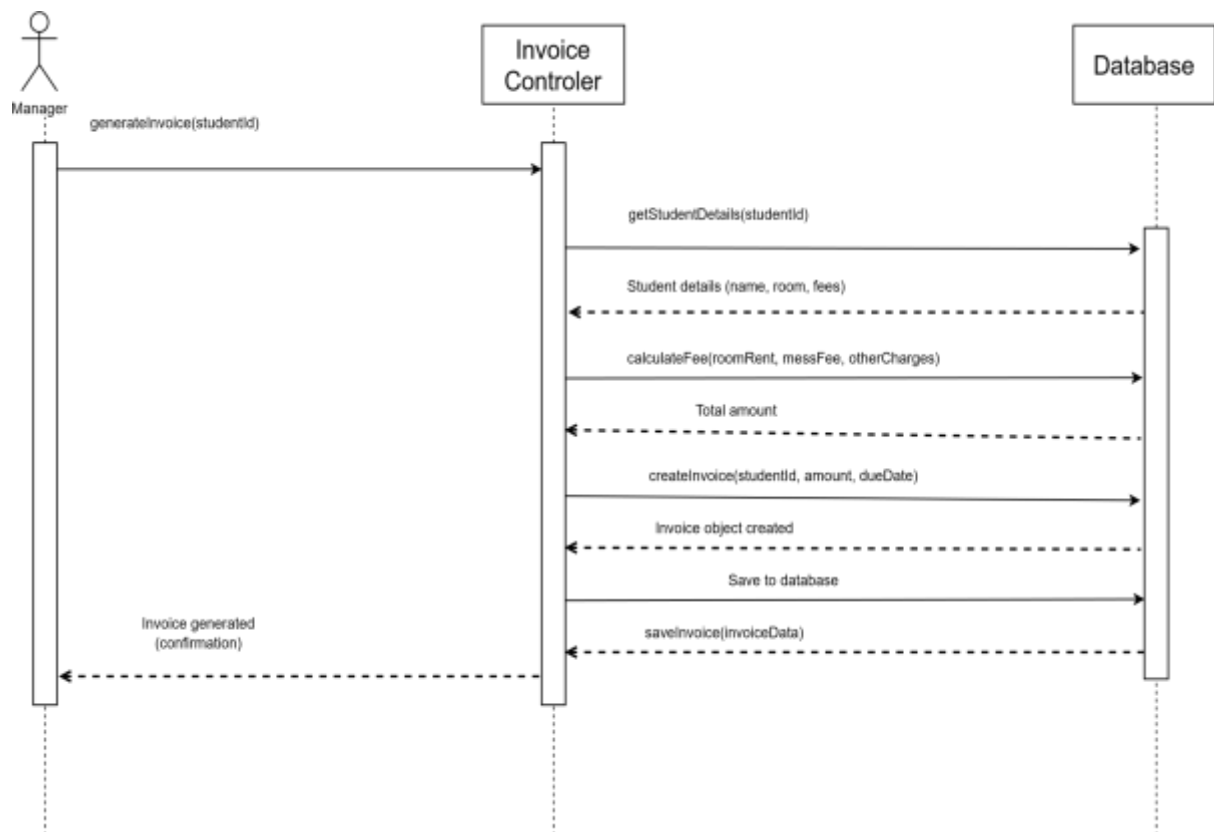


Fig-6: Sequence Diagram: Invoice

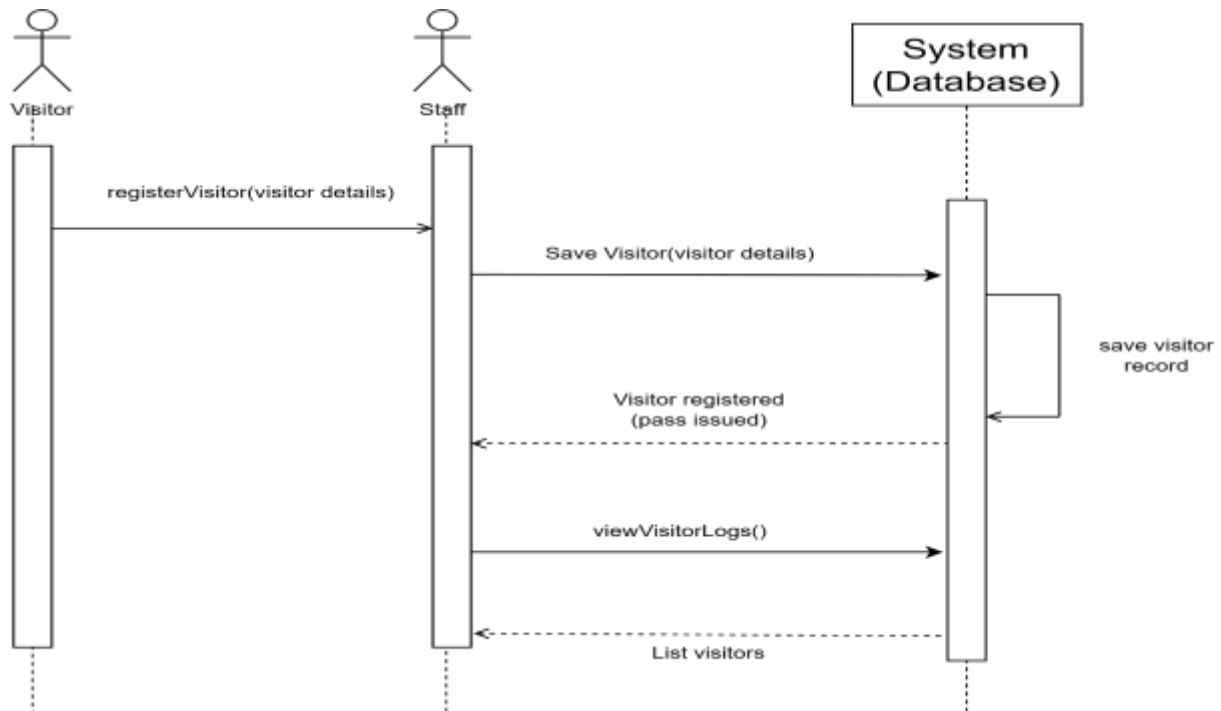


Fig-7: Sequence Diagram: Visitors

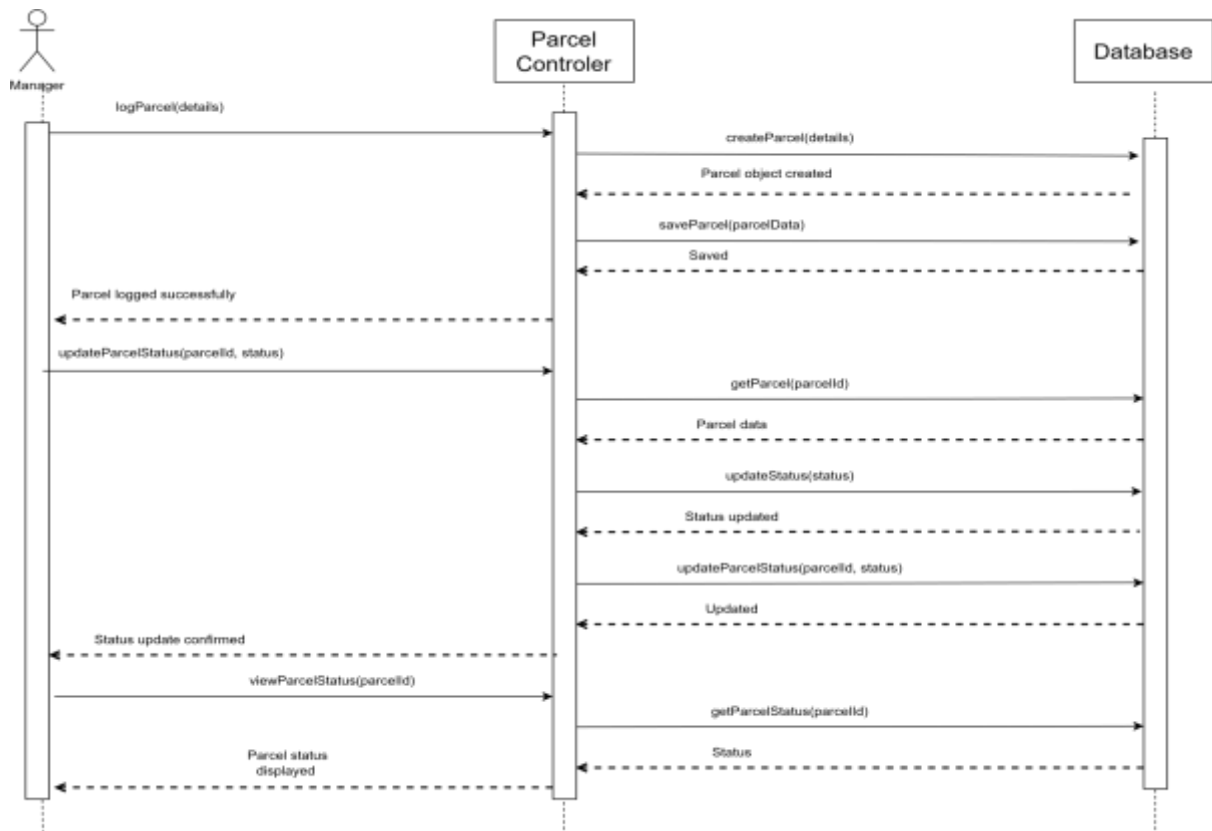


Fig-8: Sequence Diagram: Parcel

8. CPM

Task description	Task ID	Dependency	Duration
Project Planning & Scope Definition	T1	None	3
Requirements Gathering	T2	T1	5
Software Requirement Specification (SRS)	T3	T2	4
Database Design & Implementation	T4	T3	4
Development Environment Setup (PHP, MySQL, XAMPP)	T5	T3	2
UI/UX Design Prototype	T6	T3	8
Authentication & Authorization Module	T7	T4, T5, T6	7
Admin Dashboard Module	T8	T6, T7	7
Student Management Module	T9	T7	7
Attendance Management Module	T10	T9	5
Fee & Invoice Management Module	T11	T10	6
Mess-Off & Menu Management Module	T12	T9	4
Visitor & Parcel Management Module	T13	T7, T8, T9	4
Complaint & Suggestion Module	T14	T8, T9	4
Frontend Integration Design	T15	T6, T8, T9	5
Testing, Security Validation & Bug Fixing	T16	T11, T12, T13, T14, T15	10
Deployment, Documentation & User Training	T17	T16	5

Table-5: Project WBS

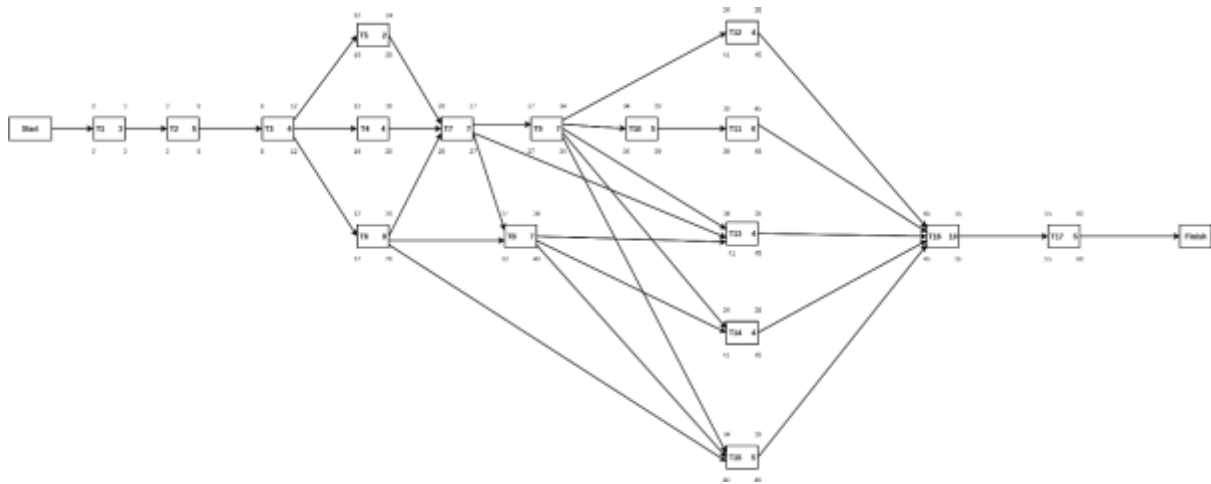


Fig-9: CPM

Critical Path, CP: T1, T2, T3, T6, T7, T9, T10, T11, T16, T17

$$= 3+5+4+8+7+7+5+6+10+5 = 60 \text{ Days}$$

Task ID	Total Float LS - ES	Free Float $\min\{ES(\text{Next Task}) - EF(\text{Current Task})\}$
T1	$0 - 0 = 0$	$3 - 3 = 0$
T2	$3 - 3 = 0$	$8 - 8 = 0$
T3	$8 - 8 = 0$	$12 - 12 = 0$
T4	$16 - 12 = 4$	$20 - 16 = 4$
T5	$18 - 12 = 6$	$20 - 14 = 6$
T6	$12 - 12 = 0$	$20 - 20 = 0$
T7	$20 - 20 = 0$	$27 - 27 = 0$
T8	$33 - 27 = 6$	$34 - 34 = 0$
T9	$27 - 27 = 0$	$34 - 34 = 0$
T10	$34 - 34 = 0$	$39 - 39 = 0$
T11	$39 - 39 = 0$	$45 - 45 = 0$
T12	$41 - 34 = 7$	$45 - 38 = 7$
T13	$41 - 34 = 7$	$45 - 38 = 7$
T14	$41 - 34 = 7$	$45 - 38 = 7$
T15	$40 - 34 = 6$	$45 - 39 = 6$
T16	$45 - 45 = 0$	$55 - 55 = 0$
T17	$55 - 55 = 0$	$60 - 60 = 0$

Table-6: Project CP, TF, FF Calculation

PERT

Task description	Task ID	Dependency	Optimistic(t_o)	ML	Pessimistic(t_p)	$t_e = (t_o + 4t_m + t_p)/6$	Variance $\sigma^2 = ((t_p - t_o)/6)^2$
Project Planning & Scope Definition	T1	None	2	3	4	3	0.11
Requirements Gathering	T2	T1	3	5	7	5	0.44
Software Requirement Specification (SRS)	T3	T2	2	4	6	4	0.44
Database Design & Implementation	T4	T3	3	4	5	4	0.11
Development Environment Setup(PHP, MySQL, XAMPP)	T5	T3	1	2	3	2	0.11
UI/UX Design Prototype	T6	T3	6	8	10	8	0.44
Authentication & Authorization Module	T7	T4, T5, T6	5	7	9	7	0.44
Admin Dashboard Module	T8	T6, T7	5	7	10	7.17	0.69
Student Management Module	T9	T7	5	7	9	7	0.44
Attendance Management Module	T10	T9	4	5	7	5.17	0.25
Fee & Invoice Management Module	T11	T10	4	6	8	6	0.44
Meta-Off & Menu Management Module	T12	T9	3	4	5	4	0.11
Visitor & Parcel Management Module	T13	T8, T9	3	4	6	4.17	0.25
Complaint & Suggestion Module	T14	T8, T9	3	4	5	4	0.11
Frontend Integration Design	T15	T6, T8, T9	4	5	7	5.17	0.25
Testing, Security Validation & Bug Fixing	T16	T11, T12, T13, T14, T15	7	10	13	10	1
Deployment, Documentation & User Training	T17	T16	3	5	7	5	0.44

Table-7: Project PERT Calculation

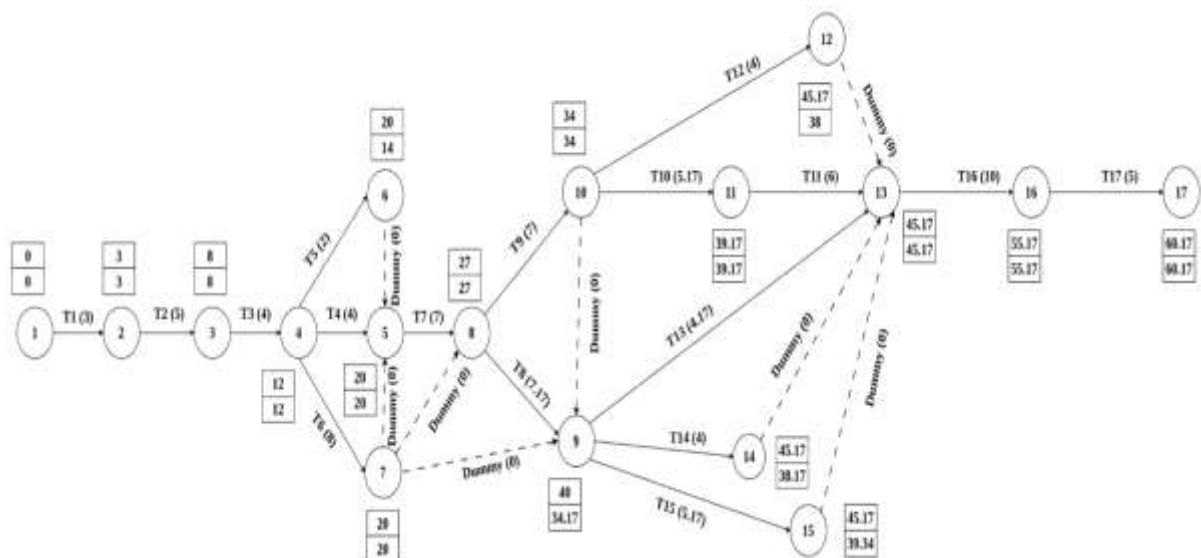


Fig-10: PERT Analysis

Expected project length: $T_1 + T_2 + T_3 + T_4 + T_6 + T_7 + T_9 + T_{10} + T_{11} + T_{16} + T_{17}$

$$= 3 + 5 + 4 + 8 + 7 + 7 + 5.17 + 6 + 10 + 5 = 60.17 \text{ days}$$

Project critical path: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 5 \rightarrow 8 \rightarrow 10 \rightarrow 11 \rightarrow 13 \rightarrow 16 \rightarrow 17$

Project length variance, $\sigma^2 = 0.11 + 0.44 + 0.44 + 0.11 + 0.44 + 0.44 + 0.25 + 0.44 + 1 + 0.44$

$$= 4.55$$

Project length Standard deviation, $\sigma = 2.13$

9. Screenshots of UI

Admin/Manager panel

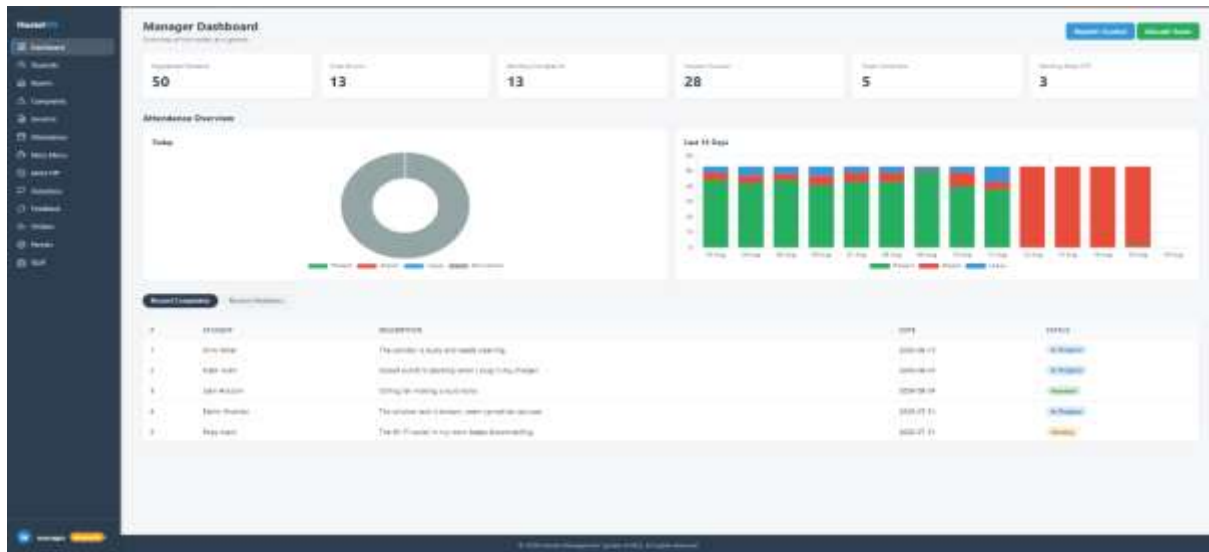


Fig-11: Dashbord

HostelMS

Add Hostel

Create a new hostel building. You can type the coordinates or pin the location on the map.

Hostel name

e.g. Boys Hostel 2

Gender

-- Select --

Location / address

e.g. East Campus (auto-filled when you drop a pin)

Geo-fence (attendance area)

Click or drag the pin on the map to set the hostel's location, or type the coordinates below. Students must be inside the blue circle to mark "Present".

Map

Latitude: e.g. 23.7685922

Longitude: e.g. 90.4254990

Attendance radius (metres): 50

Radius slider: [Slider]

Add Hostel

Fig-12: Hostel with Location

Dashboard

Students

Teachers

Classrooms

Exercises

Assignments

Class History

Attendance

Exercises

Feedback

Notes

Settings

Register Student

Fill in the details of the student you want to register.

Dashboard

Students

Teachers

Classrooms

Exercises

Assignments

Class History

Attendance

Exercises

Feedback

Notes

Settings

Students

View a list of all registered students.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

[illegible]

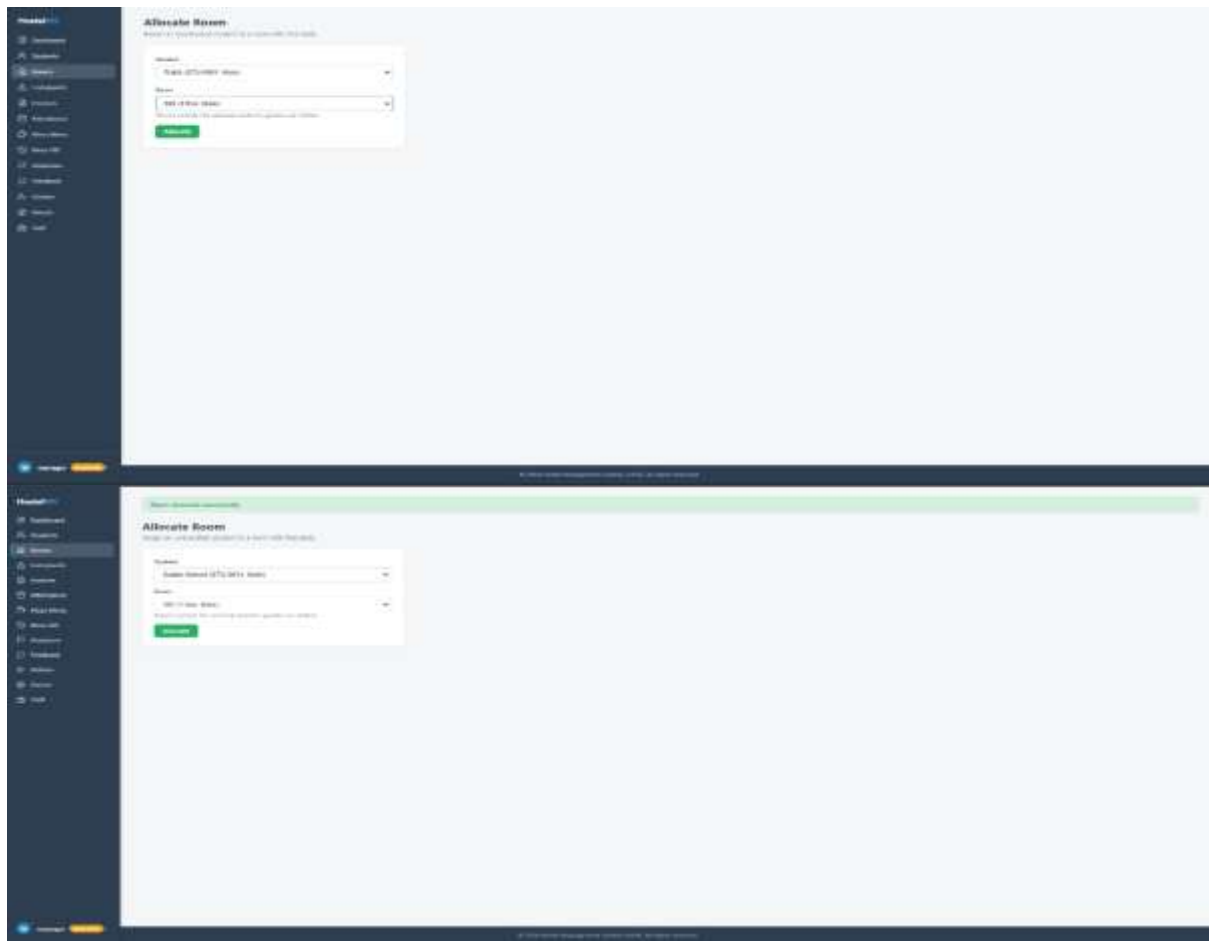


Fig-15: Room Allocation

Invoices

Summary by Type

Invoice Type	Invoice Count	Invoice Total
Room 101	17990.0075	17990.0075
Room 102	8000.0075	8000.0075
Room 103	1900.0075	1900.0075
Room 104	790.0075	790.0075
Room 105	400.0075	400.0075

Generate Invoices

Room 101

Room 102

Room 103

Room 104

Room 105

Room 106

Room 107

Room 108

Room 109

Room 110

Room 111

Room 112

Room 113

Room 114

Room 115

Room 116

Room 117

Room 118

Room 119

Room 120

Room 121

Room 122

Room 123

Room 124

Room 125

Room 126

Room 127

Room 128

Room 129

Room 130

Room 131

Room 132

Room 133

Room 134

Room 135

Room 136

Room 137

Room 138

Room 139

Room 140

Room 141

Room 142

Room 143

Room 144

Room 145

Room 146

Room 147

Room 148

Room 149

Room 150

Room 151

Room 152

Room 153

Room 154

Room 155

Room 156

Room 157

Room 158

Room 159

Room 160

Room 161

Room 162

Room 163

Room 164

Room 165

Room 166

Room 167

Room 168

Room 169

Room 170

Room 171

Room 172

Room 173

Room 174

Room 175

Room 176

Room 177

Room 178

Room 179

Room 180

Room 181

Room 182

Room 183

Room 184

Room 185

Room 186

Room 187

Room 188

Room 189

Room 190

Room 191

Room 192

Room 193

Room 194

Room 195

Room 196

Room 197

Room 198

Room 199

Room 200

Room 201

Room 202

Room 203

Room 204

Room 205

Room 206

Room 207

Room 208

Room 209

Room 210

Room 211

Room 212

Room 213

Room 214

Room 215

Room 216

Room 217

Room 218

Room 219

Room 220

Room 221

Room 222

Room 223

Room 224

Room 225

Room 226

Room 227

Room 228

Room 229

Room 230

Room 231

Room 232

Room 233

Room 234

Room 235

Room 236

Room 237

Room 238

Room 239

Room 240

Room 241

Room 242

Room 243

Room 244

Room 245

Room 246

Room 247

Room 248

Room 249

Room 250

Room 251

Room 252

Room 253

Room 254

Room 255

Room 256

Room 257

Room 258

Room 259

Room 260

Room 261

Room 262

Room 263

Room 264

Room 265

Room 266

Room 267

Room 268

Room 269

Room 270

Room 271

Room 272

Room 273

Room 274

Room 275

Room 276

Room 277

Room 278

Room 279

Room 280

Room 281

Room 282

Room 283

Room 284

Room 285

Room 286

Room 287

Room 288

Room 289

Room 290

Room 291

Room 292

Room 293

Room 294

Room 295

Room 296

Room 297

Room 298

Room 299

Room 300

Room 301

Room 302

Room 303

Room 304

Room 305

Room 306

Room 307

Room 308

Room 309

Room 310

Room 311

Room 312

Room 313

Room 314

Room 315

Room 316

Room 317

Room 318

Room 319

Room 320

Room 321

Room 322

Room 323

Room 324

Room 325

Room 326

Room 327

Room 328

Room 329

Room 330

Room 331

Room 332

Room 333

Room 334

Room 335

Room 336

Room 337

Room 338

Room 339

Room 340

Room 341

Room 342

Room 343

Room 344

Room 345

Room 346

Room 347

Room 348

Room 349

Room 350

Room 351

Room 352

Room 353

Room 354

Room 355

Room 356

Room 357

Room 358

Room 359

Room 360

Room 361

Room 362

Room 363

Room 364

Room 365

Room 366

Room 367

Room 368

Room 369

Room 370

Room 371

Room 372

Room 373

Room 374

Room 375

Room 376

Room 377

Room 378

Room 379

Room 380

Room 381

Room 382

Room 383

Room 384

Room 385

Room 386

Room 387

Room 388

Room 389

Room 390

Room 391

Room 392

Room 393

Room 394

Room 395

Room 396

Room 397

Room 398

Room 399

Room 400

Room 401

Room 402

Room 403

Room 404

Room 405

Room 406

Room 407

Room 408

Room 409

Room 410

Room 411

Room 412

Room 413

Room 414

Room 415

Room 416

Room 417

Room 418

Room 419

Room 420

Room 421

Room 422

Room 423

Room 424

Room 425

Room 426

Room 427

Room 428

Room 429

Room 430

Room 431

Room 432

Room 433

Room 434

Room 435

Room 436

Room 437

Room 438

Room 439

Room 440

Room 441

Room 442

Room 443

Room 444

Room 445

Room 446

Room 447

Room 448

Room 449

Room 450

Room 451

Room 452

Room 453

Room 454

Room 455

Room 456

Room 457

Room 458

Room 459

Room 460

Room 461

Room 462

Room 463

Room 464

Room 465

Room 466

Room 467

Room 468

Room 469

Room 470

Room 471

Room 472

Room 473

Room 474

Room 475

Room 476

Room 477

Room 478

Room 479

Room 480

Room 481

Room 482

Room 483

Room 484

Room 485

Room 486

Room 487

Room 488

Room 489

Room 490

Room 491

Room 492

Room 493

Room 494

Room 495

Room 496

Room 497

Room 498

Room 499

Room 500

Room 501

Room 502

Room 503

Room 504

Room 505

Room 506

Room 507

Room 508

Room 509

Room 510

Room 511

Room 512

Room 513

Room 514

Room 515

Room 516

Room 517

Room 518

Room 519

Room 520

Room 521

Room 522

Room 523

Room 524

Room 525

Room 526

Room 527

Room 528

Room 529

Room 530

Room 531

Room 532

Room 533

Room 534

Room 535

Room 536

Room 537

Room 538

Room 539

Room 540

Room 541

Room 542

Room 543

Room 544

Room 545

Room 546

Room 547

Room 548

Room 549

Room 550

Room 551

Room 552

Room 553

Room 554

Room 555

Room 556

Room 557

Room 558

Room 559

Room 560

Room 561

Room 562

Room 563

Room 564

Room 565

Room 566

Room 567

Room 568

Room 569

Room 570

Room 571

Room 572

Room 573

Room 574

Room 575

Room 576

Room 577

Room 578

Room 579

Room 580

Room 581

Room 582

Room 583

Room 584

Room 585

Room 586

Room 587

Room 588

Room 589

Room 590

Room 591

Room 592

Room 593

Room 594

Room 595

Room 596

Room 597

Room 598

Room 599

Room 600

Room 601

Room 602

Room 603

Room 604

Room 605

Room 606

Room 607

Room 608

Room 609

Room 610

Room 611

Room 612

Room 613

Room 614

Room 615

Room 616

Room 617

Room 618

Room 619

Room 620

Room 621

Room 622

Room 623

Room 624

Room 625

Room 626

Room 627

Room 628

Room 629

Room 630

Room 631

Room 632

Room 633

Room 634

Room 635

Room 636

Room 637

Room 638

Room 639

Room 640

Room 641

Room 642

Room 643

Room 644

Room 645

Room 646

Room 647

Room 648

Room 649

Room 650

Room 651

Room 652

Room 653

Room 654

Room 655

Room 656

Room 657

Room 658

Room 659

Room 660

Room 661

Room 662

Room 663

Room 664

Room 665

Room 666

Room 667

Room 668

Room 669

Room 670

Room 671

Room 672

Room 673

Room 674

Room 675

Room 676

Room 677

Room 678

Room 679

Room 680

Room 681

Room 682

Room 683

Room 684

Room 685

Room 686

Room 687

Room 688

Room 689

Room 690

Room 691

Room 692

Room 693

Room 694

Room 695

Room 696

Room 697

Room 698

Room 699

Room 700

Room 701

Room 702

Room 703

Room 704

Room 705

Room 706

Room 707

Room 708

Room 709

Room 710

Room 711

Room 712

Room 713

Room 714

Room 715

Room 716

Room 717

Room 718

Room 719

Room 720

Room 721

Room 722

Room 723

Room 724

Room 725

Room 726

Room 727

Room 728

Room 729

Room 730

Room 731

Room 732

Room 733

Room 734

Room 735

Room 736

Room 737

Room 738

Room 739

Room 740

Room 741

Room 742

Room 743

Room 744

Room 745

Room 746

Room 747

Room 748

Room 749

Room 750

Room 751

Room 752

Room 753

Room 754

Room 755

Room 756

Room 757

Room 758

Room 759

Room 760

Room 761

Room 762

Room 763

Room 764

Room 765

Room 766

Room 767

Room 768

Room 769

Room 770

Room 771

Room 772

Room 773

Room 774

Room 775

Room 776

Room 777

Room 778

Room 779

Room 780

Room 781

Room 782

Room 783

Room 784

Room 785

Room 786

Room 787

Room 788

Room 789

Room 790

Room 791

Room 792

Room 793

Room 794

Room 795

Room 796

Room 797

Room 798

Room 799

Room 800

Room 801

Room 802

Room 803

Room 804

Room 805

Room 806

Room 807

Room 808

Room 809

Room 810

Room 811

Room 812

Room 813

Room 814

Room 815

Room 816

Room 817

Room 818

Room 819

Room 820

Room 821

Room 822

Room 823

Room 824

Room 825

Room 826

Room 827

Room 828

Room 829

Room 830

Room 831

Room 832

Room 833

Room 834

Room 835

Room 836

Room 837

Room 838

Room 839

Room 840

Room 841

Room 842

Room 843

Room 844

Room 845

Room 846

Room 847

Room 848

Room 849

Room 850

Room 851

Room 852

Room 853

Room 854

Room 855

Room 856

Room 857

Room 858

Room 859

Room 860

Room 861

Room 862

Room 863

Room 864

Room 865

Room 866

Room 867

Room 868

Room 869

Room 870

Room 871

Room 872

Room 873

Room 874

Room 875

Room 876

Room 877

Room 878

Room 879

Room 880

Room 881

Room 882

Room 883

Room 884

Room 885

Room 886

Room 887

Room 888

Room 889

Room 890

Room 891

Room 892

Room 893

Room 894

Room 895

Room 896

Room 897

Room 898

Room 899

Room 900

Room 901

Room 902

Room 903

Room 904

Room 905

Room 906

Room 907

Room 908

Room 909

Room 910

Room 911

Room 912

Room 913

Room 914

Room 915

Room 916

Room 917

Room 918

Room 919

Room 920

Room 921

Room 922

Room 923

Room 924

Room 925

Room 926

Room 927

Room 928

Room 929

Room 930

Room 931

Room 932

Room 933

Room 934

Room 935

Room 936

Room 937

Room 938

Room 939

Room 940

Room 941

Room 942

Room 943

Room 944

Room 945

Room 946

Room 947

Room 948

Room 949

Room 950

Room 951

Room 952

Room 953

Room 954

Room 955

Room 956

Room 957

Room 958

Room 959

Room 960

Room 961

Room 962

Room 963

Room 964

Room 965

Room 966

Room 967

Room 968

Room 969

Room 970

Room 971

Room 972

Room 973

Room 974

Room 975

Room 976

Room 977

Room 978

Room 979

Room 980

Room 981

Room 982

Room 983

Room 984

Room 985

Room 986

Room 987

Room 988

Room 989

Room 990

Room 991

Room 992

Room 993

Room 994

Room 995

Room 996

Room 997

Room 998

Room 999

Room 1000

Fig-16: Invoices generate

Student/Panel

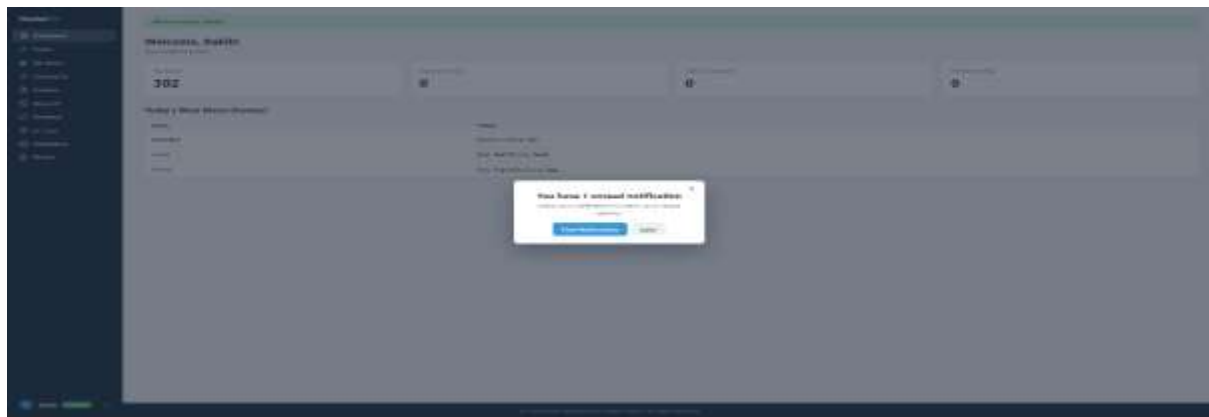


Fig-17: Dashboard

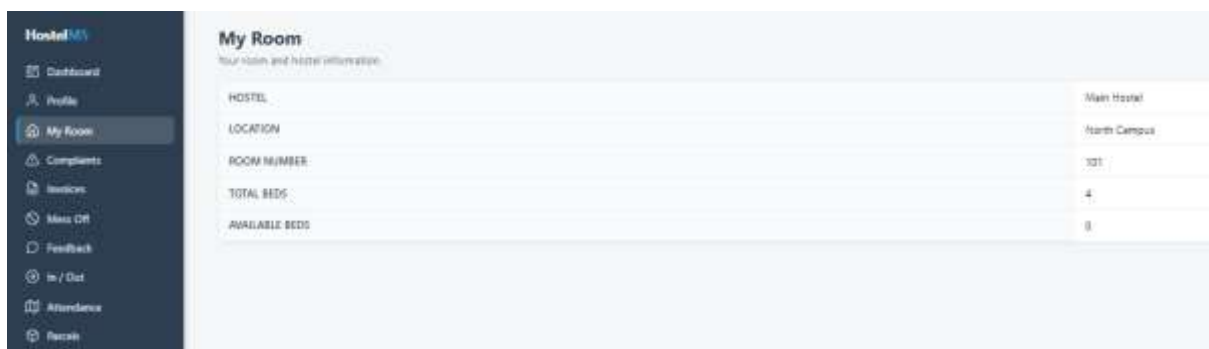


Fig-18: Allocated Room

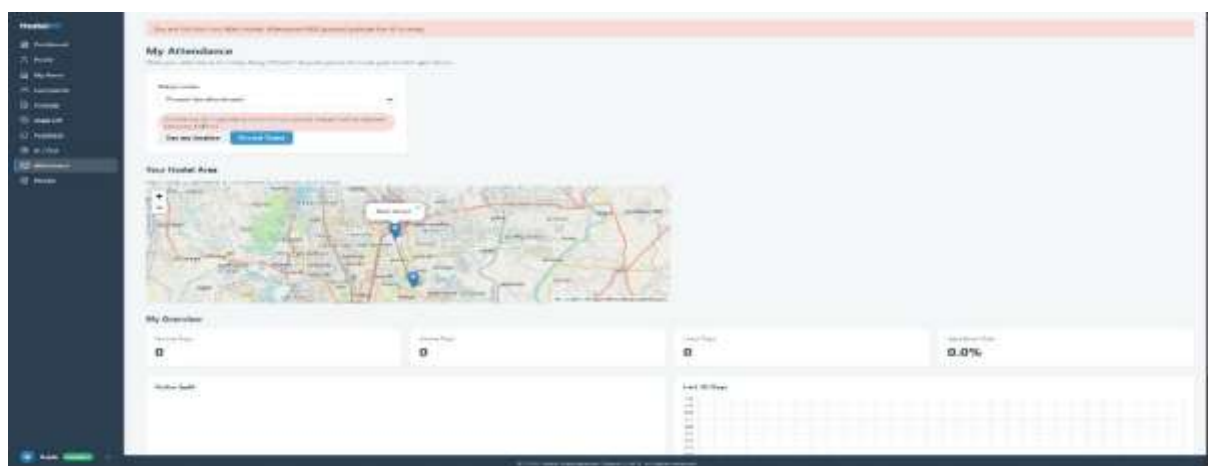


Fig-19: Attendance

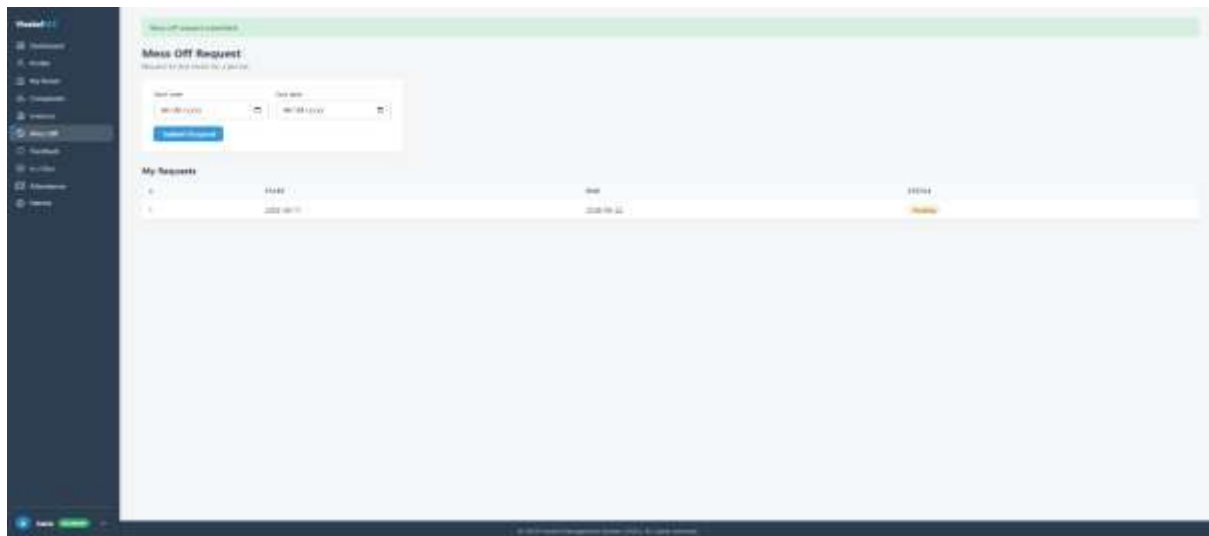


Fig-20: Mess-off Request



Fig-21: Register Complaints



Fig-22: In and Out History

Staff Panel

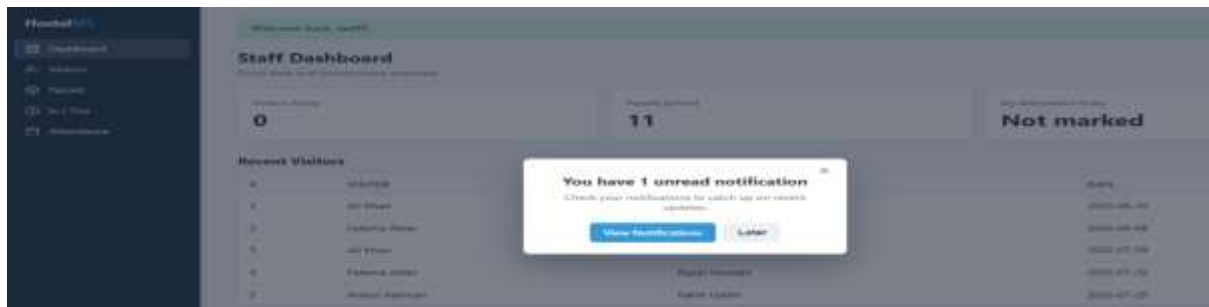


Fig-23: Dashboard

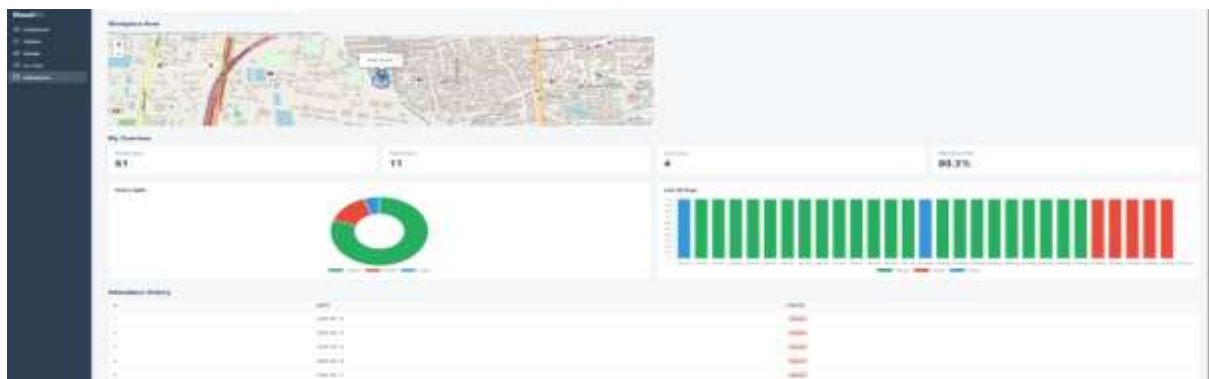


Fig-24: Attendance

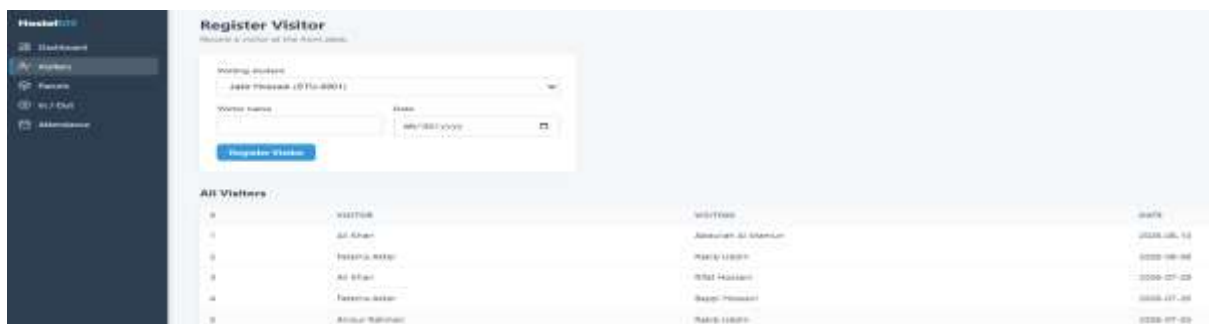


Fig-25: Register visitor

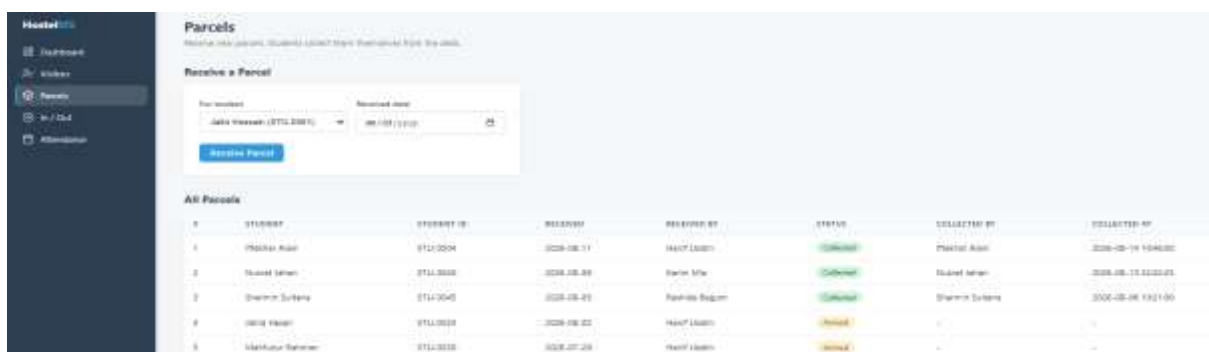


Fig-26: parcel collection

Conclusion

The Hostel Management System successfully addresses key challenges of manual hostel administration through a centralized web-based platform for Managers, Staff, and Students. Features such as geo-fenced attendance, automated invoices, notifications, and reporting improve administrative efficiency, transparency, and accessibility. The project also demonstrates the feasibility of developing and deploying a full-featured system using open-source technologies within a limited budget and timeframe.

Future recommendations:

- Set up automated daily database backups.
- Add a 'Forgot Password' feature for user convenience.
- Add pagination for large datasets (attendance, invoices).
- Provide a printable PDF attendance report per student.

References

- [1] A. Awoniyi, “Development of a Web-Based Hostel Management System,” *British Journal of Computer, Networking and Information Technology*, vol. 8, no. 1, pp. 30–41, 2025.
- [2] N. Yusof, N. H. A. Saiful Nizan, N. K. Ibrahim, Z. Othman, and A. S. Yusof, “StaySmart: A Web-Based Hostel Management System for Student Accommodation,” *International Journal of Research and Innovation in Social Science*, vol. 10, no. 3, pp. 436–444, 2026.
- [3] R. S. B. N. S. V. A. Lakkimsetti et al., “Design and Implementation of a Secure Web-Based Hostel and PG Management System Using Open-Source Technologies,” *International Journal for Research in Applied Science and Engineering Technology*, 2026.
- [4] “Attendance Management System Using Geofencing Technology,” in *Proc. 1st Int. Conf. Pioneering Developments in Computer Science & Digital Technologies (IC2SDT)*, Delhi, India, Aug. 2024, doi: 10.1109/IC2SDT62152.2024.10696345.
- [5] J. F. Barkley, D. R. Kuhn, L. S. Rosenthal, M. W. Skall, and A. V. Cincotta, “Role-Based Access Control for the Web,” in *CALS Expo International and 21st Century Commerce*, Long Beach, CA, USA, 1998.
- [6] OWASP Foundation, “SQL Injection Prevention Cheat Sheet,” OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html